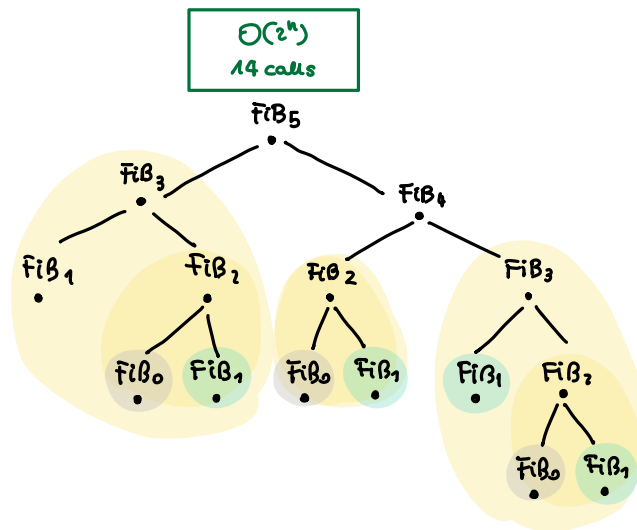


limiti della ricorsione : ipotesi di **INDIPENDENZA** dei **SOTIOPROBLEMI**

$\Rightarrow$ memoria $\hat{=}$, tempo $\hat{=}$



(A) APPLICABILITÀ della P.D.

Problemi di ottimizzazione

1) Esistenza di una **SOTTOSTRUTTURA OTTIMA** = "APPLICABILITÀ"

- Memorizzo la sottostruttura e ottengo il valore della sol ottima
- Ricostruisco per ricombinazione la sol ottima

2) Molti **SOTIOPROBLEMI** in COMUNE =

- Pro rispetto al **DIVIDE et IMPERA** che assume l'indipendenza dei sottoproblemi

3) **N° SOTOP. POLINOMIALE** "CONVENIENZA"

$$\leq T(n) = O(n \cdot c)$$

- n = num sottop
- c = costo di ricombinazione

delle sol ottime

4) Approccio **BOTTOM-UP** :

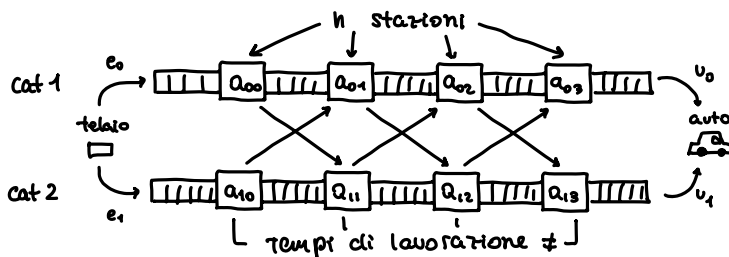
- Devo poter partire dai problemi elementari

5) **ITERAZIONE** :

- ogni sottop viene risolto dal basso 1 e 1 sola volta
compiendo a ogni passo una decisione

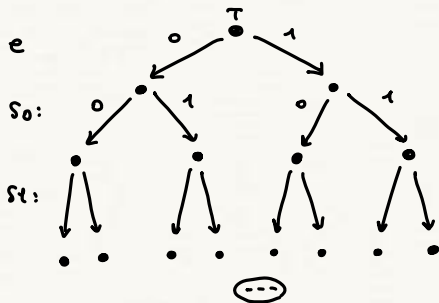
③ PROBLEMI RISOLVIBILI con P.D.

• **CATENE di MONTAGGIO**



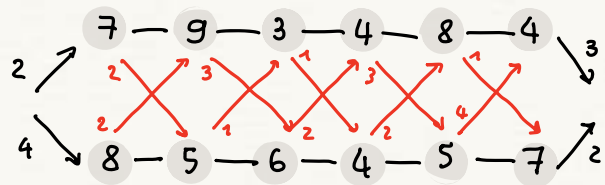
OBIETTIVO
tempo minimo

MODELO : D&I con PRINCIPIO di MULTIPLICAZIONE



Albero binario completo
 $T(n) = \Theta(2^n)$

MODELO : P.D.



	0	1	2	3	4	5	6
t_min 0	9	18	20	24	32	35	38
1	12	16	22	25	30	37	39

$9+9 < 12+2+9$
 $9+2+5 < 12+5$

	0	1	2	3	4	5	6
prov 0		0	1	0	0	1	0
1		0	1	0	1	1	0

RICOSTRUZIONE
sol ottima

PROVENIENZA

$$T(n) = \Theta(n \cdot c) = \Theta(n)$$

// CALCOLO del TEMPO MINIMO

```
int assembly_line_DP (temp, t, temp,
                      int ** t_min, int ** prov, int n) {
```

// PROB. ELEMENTARE

```
t_min[0][0] = temp.e[0] + temp.a[0][0];
t_min[1][0] = temp.e[1] + temp.a[1][0];
```

// ITERAZIONE

```
int toa, tob, tia, tib;
```

```
for (int i=1; i<n; i++) {
```

```
    * toa = t_min[0][i-1] + temp.a[0][i];
    tob = t_min[1][i-1] + temp.t[1][i-1] + temp.a[1][i];
```

```
    if (toa <= tob) { // resto in 0 → 0
```

```
        t_min[0][i] = toa;    prov[0][i] = 0;
```

```
    } else { // cambio 0 → 1
```

```
        t_min[0][i] = tob;    prov[0][i] = 1;
    }
```

⊗ stema cosa per CATENA 1

// RESTITUISCO IL VAL OTTIMO

```
int res0 = t_min[0][n-1] + temp.x[0];
```

```
int res1 = t_min[1][n-1] + temp.x[1];
```

```
if (res0 <= res1) {
```

```
    prov[0][n] = prov[1][n] = 0;
```

```
    return res0;
```

```
}
```

```
prov[0][n] = prov[1][n] = 1;
```

```
return res1;
```

```
}
```

// RICOSTRUZIONE della SOL-BEST

```
void SolDisplay_R (int ** prov, int i, int n) {
```

```
    if (n==0) return;
```

```
    SolDisplay_R (prov, prov[i][n-1], n-1);
```

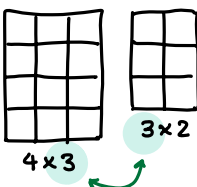
```
    printf("stazione %d - catena %d\n", n-1, i);
```

```
    return;
```

```
}
```

• PARENTEZZAZIONE OTTIMA

PRODOTTI $M \times N$

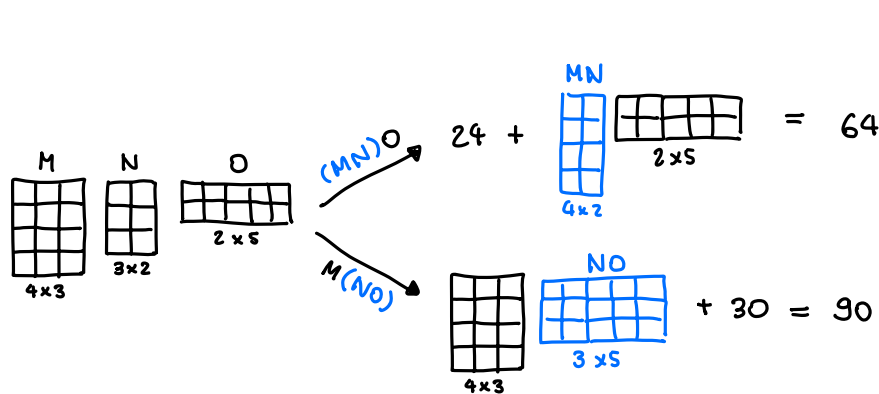


$$\Rightarrow \text{n° di moltiplicazioni} = 4 \times 3 \times 2 = 24$$

PARENTEZZAZIONE

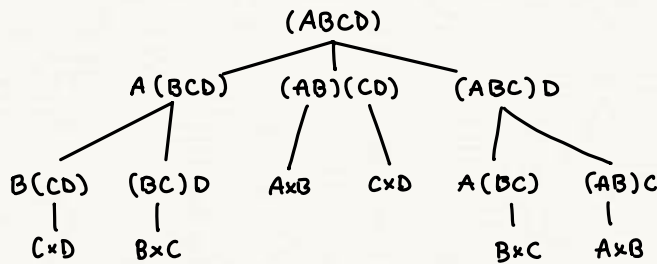
Definisce l'ordine di applicazione delle operazioni

$A_1 \times A_2 \times A_3 \times A_4$ ha 5 parentesi. possibili



$(A_1 (A_2 (A_3 A_4)))$
 $(A_1 ((A_2 A_3) A_4))$
 $((A_1 A_2) (A_3 A_4))$
 $((A_1 (A_2 A_3)) A_4)$
 $((A_1 A_2) A_3) A_4$

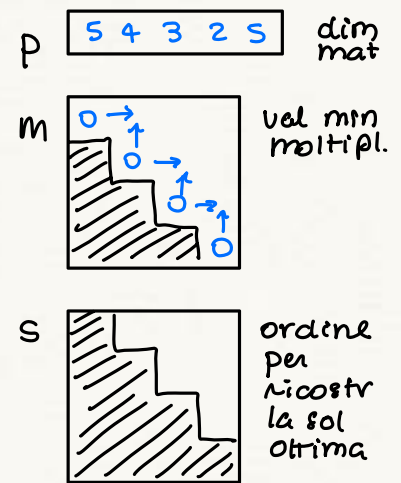
MODELLO RICORSIVO



$$m[i, j] = \begin{cases} 0 & \text{se } i=j \\ \min_{1 \leq k \leq j} \{ m[i, k] + m[k+1, j] + p_{i-1} p_k p_j \} & \text{se } i < j \end{cases}$$

$$T(n) = \Omega(2^n)$$

MODELLO P.D.

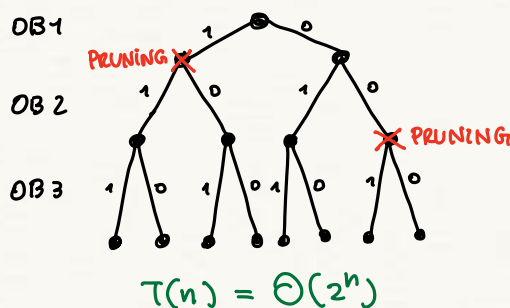


$$T(n) = \Theta(n^3)$$

- lis "Longest Increasing Sequence"

$$\text{seq} = \{6, 3, 5, 2, 7, 8, 1\} \rightarrow 3, 5, 7, 8$$

MODELLO: powerset D'



$$T(n) = \Theta(2^n)$$

MODELLO: P.D.

	0	1	2	3	4	5	6
val	6	3	5	2	7	8	1
len	1	1	2	1	3	4	1
father	-1	-1	1	-1	2	4	-1

← RILISTE
 SOL. OTTIMA

$$T(n) = \Theta(n^2 \cdot c) = \Theta(n^2)$$


```
void LIS_DP (int * val, int N) {
```

```
    int max = 1, last = 0;
    int * len = malloc (N * sizeof(int));
    int * fath = malloc (N * sizeof(int));
    for (int i = 0; i < N; i++)
        fath[i] = -1;
```

```
    // PROBLEMA ELEMENTARE
```

```
    len[0] = 1;
    fath[0] = -1;
```

```
    // ITERAZIONE: CALCOLO VAL BEST
```

```
    for (int i = 1; i < N; i++) {
```

```
        len[i] = 1;
        fath[i] = -1;
```

```
        for (int j = 0; j < i; j++)
            if ((val[j] < val[i]) && (len[i] < 1 + len[j])) {
                len[i] = 1 + len[j];
                fath[i] = j;
            }
```

```
        if (len[i] >= max) {
            max = len[i];
            last = i;
        }
```

```
    // RICOSTRUZIONE SOL
```

```
    printf("One of the LIS of len %d is: ", max);
    LISdisplay_R(val, fath, last);
```

```
void LISdisplay_R (int * val, int * fath, int last) {
```

```
    if (fath[last] == -1) {
        printf("%d ", val[last]);
        return;
    }
```

```
    LISdisplay_R (val, fath, fath[last]);
    printf("%d ", val[last]);
```

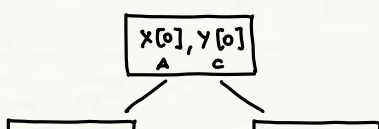
• LCS "Longest Common Sequence"

X = {A, C, G, C, T, A, C}

Y = {C, T, G, A, C, A}

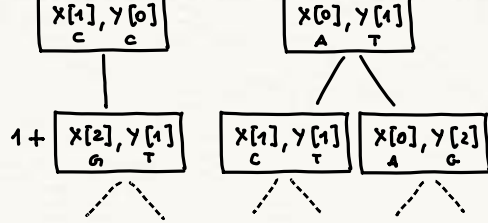
→ C G C A

MODELLO RICORSIVO



```
void LCS_R (i, j) {
```

```
    if (x[i] == '\0' || y[j] == '\0')
        return 0;
```



$$T(n) = O(2^n)$$

```

if (X[i] == Y[j])
    return 1 + LCS-R(i+1, j+1);
else
    return max(LCS-R(i+1, j), LCS-R(i, j+1));
}

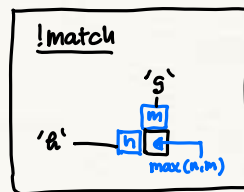
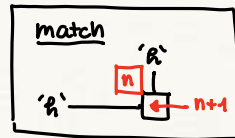
```

MODELLO: P.D.

		A	C	G	C	T	A	C	
		0	1	2	3	4	5	6	7
0	0	0	0	0	0	0	0	0	0
C	1	0	0	1	1	1	1	1	1
T	2	0	0	1	1	1	2	2	2
G	3	0	0	1	2	2	2	2	2
A	4	0	1	1	2	2	2	3	3
C	5	0	1	2	2	3	3	3	4
A	6	0	1	2	2	3	3	4	4

C G C A

$$T(n) = O(n \cdot m)$$



RICOSTRUZIONE
SOL OTTIMA

```

void LCS-DP(char *X, char *Y) {

```

```

    m = strlen(X);
    n = strlen(Y);
    int **mat = malloc((n+1) * sizeof(int *));
    for (int i = 0; i < (n+1); i++)
        mat[i] = calloc(m+1, sizeof(int));

```

// ITERAZIONE: calcolo val max

```

for (int i = 1; i <= m; i++) {
    for (int j = 1; j <= n; j++) {
        if (X[i-1] == Y[j-1])
            mat[i][j] = mat[i-1][j-1] + 1;
        else
            mat[i][j] = max(mat[i-1][j], mat[i][j-1]);
    }
}

```

// RICOSTRUZIONE SOL:

```

    SOLdisplay-R(mat, m, n, X, Y) {

```

```

void SOLdisplay-R(int **mat, int i, int j) {

```

```

    if ((i == 0) || (j == 0)) return;

```

```

    if (mat[i][j] == mat[i-1][j]) {
        SOLdisplay-R(mat, i-1, j);
        return;
    }

```

```

    if (mat[i][j] == mat[i][j-1]) {

```

```

    if (mat[i][j] == mat[i][j-1])
    {
        Sol display - R(mat, i, j-1);
        return;
    }
    Sol display - R(mat, i-1, j-1);
    printf("%c ", Y[j-1]);
}

```

• 7AINO DISCRETO

$n = 4$ val

1	2	5	6
---	---	---	---

 $cap = 8$ wt

2	3	4	5
---	---	---	---

$\hookrightarrow mat[4+1][8+1]$

$T(n) = \mathcal{O}(n \cdot cap)$

val	wt	0	1	2	3	4	5	6	7	8
0	0	0	0	0	0	0	0	0	0	0
1	2	0	0	1	1	1	1	1	1	1
2	3	0	0	1	2	2	3	3	3	3
5	4	0	0	1	2	5	5	6	7	7
6	5	0	0	1	2	5	6	6	7	8

(2,3) (6,5) Ric. sol. ott.

$wt > j$

$wt \leq j$

```

void knapsack-DP(int *val, int *wt, int N, int cap) {

```

```

    int **mat = malloc((N+1) * sizeof(int *));
    for (int i = 0; i < (N+1); i++)
        mat[i] = calloc((cap+1) * sizeof(int));

```

// ITERAZIONE: calcolo val ottime

```

    for (int i = 1; i <= N; i++) {
        for (int j = 1; j <= cap; j++) {
            if (wt[i-1] <= j)
                mat[i][j] = max(val[i-1] + mat[i-1][j - wt[i-1]],
                                mat[i-1][j]);
            else
                mat[i][j] = mat[i-1][j];
        }
    }

```

// RICOSTRUZIONE SOL

```

    Sol display - R(val, wt, mat, N, cap);
}

```

```

void Sol display - R(..., int i, int j) {

```

```

    if (i == 0 || j == 0) return;

```

```

    if (mat[i][j] == mat[i-1][j]) {

```

```

        Sol display - R(..., i-1, j);
        return;
    }

```

```

    else {

```

```

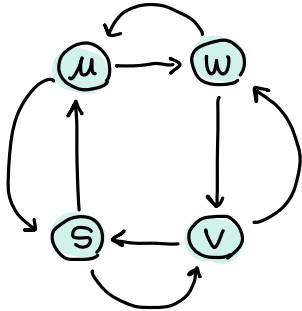
        Sol display - R(..., i-1, j - wt[i-1]);
        printf("%d ", val[i-1]);
        return;
    }
}

```

$j - wt$

③ ESISTENZA SOTTOSTRUTTURA OTTIMA

G ORIENTATO: CAMMINO SEMPLICE max/min DA (u) A (v) .



(v)

per CAMMINO MIN =

se sottop 1,2 NON ottimo

$\Rightarrow \exists$ sottop 1,2' ottimo

$\Rightarrow p$ NON ottimo (dim. per ASSURDO)

(x)

per CAMMINO MAX:

se sottop 1 $u \rightarrow w$ NON max

$\Rightarrow \max u \rightarrow s \rightarrow v \rightarrow w \equiv$ sottop di $u \rightarrow w$

se sottop 2 $w \rightarrow v$ NON max

$\Rightarrow \max w \rightarrow u \rightarrow s \rightarrow v \equiv$ sottop di $w \rightarrow v$

$\Rightarrow p$ SAREBBE OTTIMO

$\Rightarrow \nexists$ sottostuttura ottima

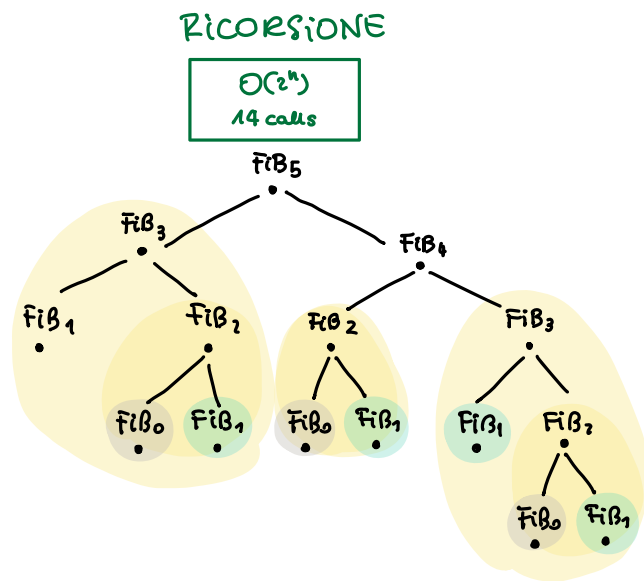
Approccio molto $\cong$ al **DIVIDE & IMPERA** ricorsivo

↳ steso modello di ricorsione

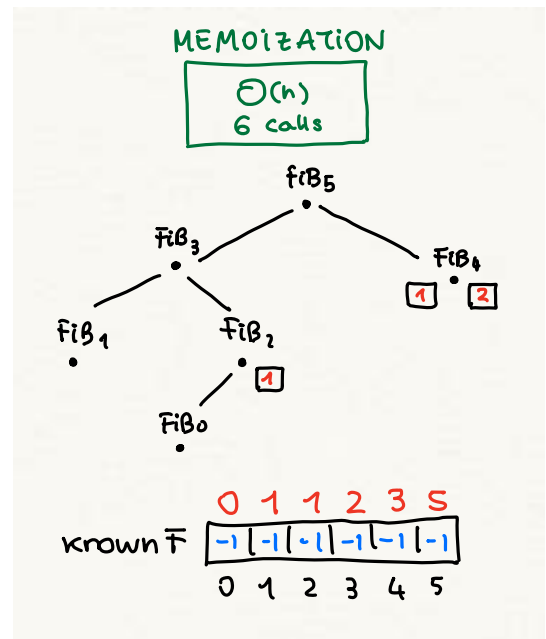
↳ **TOP-DOWN**

Differenza: **MEMORIZZAZIONE** delle **SOL ai SOTTOPROBLEMI**

- non più considerati indipendenti
- si risolvono 1 e 1 sola volta



$\Rightarrow$



nel codice della ricorsione
aggiungo il controllo:

```

if (knownF[n] != -1) {
    * ↓ return knownF[n];
}
  
```

$\oplus(1)$

 Si può applicare dove **TANTI SOTTOPROBLEMI RIPETUTI**!

CAPITOLO 10

PARADIGMA GREEDY

Per PROB. di OTTIMIZZAZIONE, alternativo a D&I e P.D.

↳ MINOR COMPLESSITA'

↳ (MA) NON sempre trova la SOL OTTIMA

- solo se questa ha sottop OTIMI LOCALMENTE

Principi

1) A ogni passo si scelgono SOL LOCALMENTE OTTIME

- sulla base di una FUNZ. di APPETIBILITA'

2) le scelte fatte NON RICONSIDERATE

- no backtrace

Funz. di Appetibilità

L'appetibilità di ogni scelta è SCELTA in PARTENZA

Può essere:

↳ NON MODIFICABILE "SELEZ. ATTIVITA'" "PRIM'S" "KRUSKAL'S"

⇒ elementi ordinati per APPETIB DECRESCENTI

⇒ scelte in sequenza

↳ MODIFICABILE "HUFFMAN" "DIJKSTRA"

⇒ priority queue

(A) PROBLEMI TIPICI

- ACTIVITY SELECTION

	0	1	2	3	4
ti	1	3	2	4	3

1 2

Attività
 tf

2	4	5	7	8
---	---	---	---	---

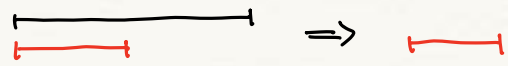
OB: MAX N° ATT COMPATIBILI

- t_i CRESCENTE? NO : inizia presto ma dura a lungo
 - l_{en} CRESCENTE? NO : ne elimina 2 lunghe compatibili
- ⇒ t_f CRESCENTE! SÌ
 $T(n) = \mathcal{O}(n)$
- le altre o si intersecano, o compatibili!
- Ottima

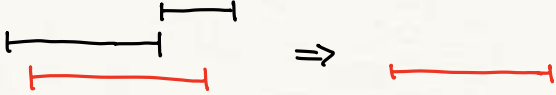
• ACTIVITY SELECTION 2

OB: SOMMA DELLE DURATE MAX

- t_f CRESCENTE : non garantisce la sol ottima



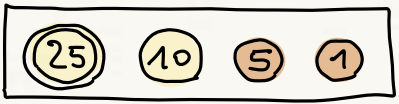
- l_{en} DECRESCENTE : nemmeno



• CAMBIA MONETE

RESTO CON MIN N° di MONETE

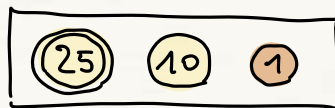
⇒ A ogni passo NON di VAL MAX ≤ RESTO



67 → 25, 42 → 25, 17 → 10, 7

ottima

$$(5), 2 \longrightarrow (1), 1 \longrightarrow (1).$$



$$30 \longrightarrow (25), 5 \longrightarrow (1)(1)(1)(1)(1)$$

non ottima

• ZAINO DISCRETO

OB: MAX val TOT

COND: $\sum wt < CAP$

$\Rightarrow$ Appetibilità:

$$\frac{val_j}{wt_j}$$

DECRESCENTI

"VALORE SPECIFICO"

	val	wt	val-sp
(1)	60	10	6
(2)	100	20	5
(3)	120	30	4

CAP
50

$$\Rightarrow (1), 40 \longrightarrow (2), 20 : val = 160$$

non ottima

• ZAINO CONTINUO

OB: MAX val TOT

COND: $\sum wt < CAP$

MA

POSSO PRENDERE UNA FRAZIONE

$\Rightarrow$ Appetibilità:

$$\frac{val_j}{wt_j}$$

DECRESCENTI

"VALORE SPECIFICO"

	val	wt	val-sp
(1)	60	10	6
(2)	100	20	5
(3)	120	30	4

CAP
50

$$\Rightarrow (1), 40 \longrightarrow (2), 20 \longrightarrow \left(\frac{2}{3}\right)(3), 0 :$$

$$val = 240$$

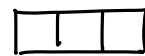
ottima

• CODICI di HUFFMAN

Simbolo $\longleftrightarrow$ stringa di bit

- CODICI a LUNGHEZZA FISSA "ASCII"

$$\text{no bit} = \log_2(N)$$



$\rightarrow$ 8 stringhe, 8 simboli

simboli ISOFREQUENTI

facile DECODIFICA

- CODICI a LUNGHEZZA VARIABILE "MORSE" $\begin{cases} A : - \\ E : . \end{cases}$

simboli con FREQUENZE $\neq$

$\Rightarrow$ PAUSA $\perp$ serve

✓ risparmio MEMORIA

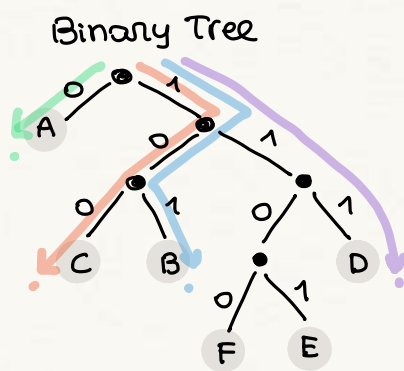
✗ difficile DECODIFICA

CODICI PREFISSI "HUFFMAN"

Codici liberi da prefisso: nessuna str di bit che cod. un simbolo e' prefisso di una second str.

$E = 00 \Rightarrow A = 001$

- CODIFICA: giustapposizione stringhe
- DECODIFICA: percorrimto albero binario



$$T(n) = \mathcal{O}(n \log_2 n)$$

cod CABD

1000101111

dec 1000101111

CABD

QUASI ADT : SYMBOL.h

```
typedef struct symbol {
    char c;
    int freq;
} sym_t;
```

ADT I CLASSE : BINARY-TREE.h

```
typedef struct binary_tree * BT;
```

BINARY-TREE.c

```
typedef struct node * link;
struct node {
```

```
    sym_t sym;
    link l;
    link r;
}
```

```
typedef struct binary_tree {
```

```
    link root;
    int cnt;
}
```

ADT I CLASSE : PQ.h

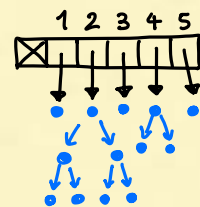
```
typedef struct pqueue * PQ;
```

PQ.c

```
struct pqueue {
```

```
    BT * heap; → malloc N
    int maxN; → N
    int heapsize; → 0
}
```

PQ ≡ HEAP



$LEFT(i) = i * 2;$
 $RIGHT(i) = i * 2 + 1;$
 $PARENT(i) = i / 2;$

TREES

sym $\square$
 l Δ r Δ

Codice

```
//inizializzo
int N = 5;
char sym[N] = {A, B, C, D, E};
PQ pq = PQinit(N);
BT bt, bt1, bt2;
```

```
//creo gli  $\mathbb{Q}$  con 1  $\emptyset \rightarrow PQ$ 
for (int i = 0; i < N; i++) {
    bt = BTinit();
    BTinsert_root(bt, sym[i]);
    PQinsert(pq, bt);
}
```

```
//a coppie: ext, join, ins
while (PQsize(pq) > 1) {
    bt1 = PQextract_min(pq);
    bt2 = PQextract_min(pq);
    bt = BTjoin(bt1, bt2);
    PQinsert(pq, bt);
}
```

```
//ultimo = huffman tree
bt = PQextract_min(pq);
BTdisplay_code(bt);
```

```
//inizial
sym_t sym[N] → char c
int freq
PQ pq = PQinit(N); "GREEDY"
BT bt;
```

```
//creo gli  $\mathbb{Q}$  con 1  $\emptyset \xrightarrow{ins} PQ$ 
for (int i = 0; i < N; i++) {
    bt = BTinit();
    BTinsert_root(bt, sym[i]);
    PQinsert(pq, bt);
}
```

```
//extr a coppie
BT bt1, bt2;
while (PQsize(pq) > 1) {
    bt1 = PQextr_min(pq);
    bt2 = PQextr_min(pq);
    bt = BTjoin(bt1, bt2);
    PQinsert(pq, bt);
}
bt = PQextr_min(pq);
BTdisplay_code(bt);
```

```
void PQinsert (PQ pq, BT bt) {  $O(\log_2 n)$ 
```

```
    // dim ++
```

```
    i = pq → heapsize ++;
```

```
    // ordinamento
```

```
    while ( i > 1 && sym lt (
        ↓ BT get-root-sym ( pq → heap [ PARENT(i) ] ),
        ↓ BT get-root-sym ( bt )
    ) ) {
```

```
        pq → heap [ i ] = pq → heap [ PARENT(i) ];
        i = PARENT(i);
    }
```

```
    // inserimento
```

```
    pq → heap [ i ] = bt;
```

```
}
```



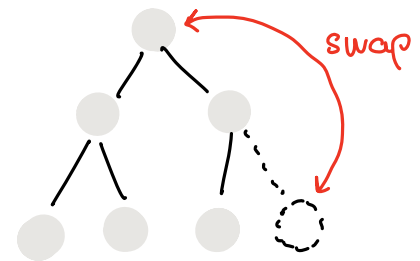
```
void PQextract Min (PQ pq) {
```

```
    PQ swap ( pq, 1, pq → heapsize );
```

```
    pq → heapsize --;
```

```
    HEAPIFY ( pq, 1 );
```

```
}
```



```
void PQ heapify (PQ pq, int i) {
```

```
    int new = i, l = LEFT(i), r = RIGHT(i);
```

```
    // CONFRONTO A SX
```

```
    if ( l <= pq → heapsize &&
```

```
        sym lt (
```

```
            ↓ BT set-root-sym ( pq → heap [ l ] ),
```

```
            ↓ BT set-root-sym ( pq → heap [ i ] )
```

```
        ) {
```

```
            new = l;
```

```
    // CONFRONTO A DX
```

```
    (--)
```

```
    new = r;
```

```
    // RICORSIONE
```

```
    if ( new != i ) {
```

```
        PQ swap ( pq, new, i );
```

```
        PQ heapify ( pq, new );
```

```
    }
```

```
}
```

```
void HEAPIFY (PQ pq, int i) {
```

```
    // MIN l e r
```

```
    if ( ITEM cmp (
```

```
void BT display-code (BT bt) {
```

```
    int l = BT get-l (bt);
```

```
    char * bits = malloc ((l+1) * sizeof(char));
```

```
    displayR (bt → root, bits, 0);
```

```
}
```

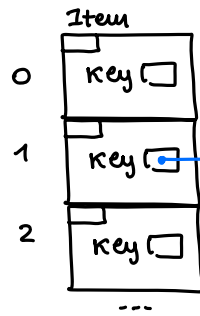
```
void displayR (link t, char *bits, int pos) {
```

```
if((t->l == NULL) && (t->r == NULL)) {  
    bits[pos] = '0';  
    printf("%c = %s\n", t->c, bits);  
    return;  
}
```

```
bits[pos] = '0';  
displayR(t->l, bits, pos+1);  
bits[pos] = '1';  
displayR(t->r, bits, pos+1);
```

3

Dizionari :



```
int HASH (key key, int maxN)
```

FUNZIONE che restituisce in $\Theta(1)$

- ✓ veloci e comode $\Theta(1)$
- ✗ spreco spazio: $n_{el} \ll \max N$
- ✗ HASH(...) imperfetta $\rightarrow$ collisioni

Ⓐ HASH FUNCTION : METODI

• MODULARE : INT

 $N = n^{\circ}$ di num int $M = \text{dim hash_tab} \rightarrow$ numero primo $h(k) \rightarrow \text{return } k \geq M;$

• MOLTIPLICATIVO : FLOAT

 $k = n^{\circ}$ float t.c. $k \in [\text{inf}, \text{sup}]$ inf $\neq$ sup $\neq$ $h(k) = (k - \text{inf}) / (\text{sup} - k) * M;$

• MODULO-MODULARE : INT

costante float : $f \in (0, 1) \rightarrow f = (\sqrt{5} - 1) / 2$ $h(k) = \lfloor kf \rfloor \geq M;$

• STRINGHE CORTE : "NOW"

 $h(\text{"now"}) = ('n' * 128^2 + 'o' * 128^1 + 'w') \% M;$

• STRINGHE LUNGHE : "VERYLONGKEY"

Col metodo prec $\Rightarrow$ non contenibile in int !!!

HORNER'S METHOD

$$h_i = (h_{i-1} * 128 + 'v') \% M;$$

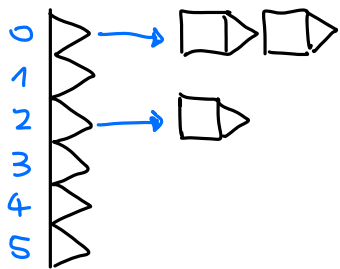
• HASH UNIVERSALE

Uso un n° pseudo casuale

↳ probabilità collisione $\approx \frac{1}{M}$

③ GESTIONE COLLISIONI

1) LINEAR CHAINING: vett. di liste



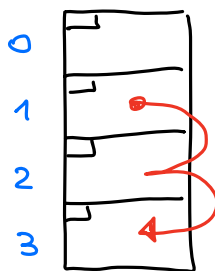
✗ MEMORIA A

✓ CANCELLAZIONE OK

2) OPEN ADDRESSING:

linear probing
quadratic probing
double hashing

 GUARDA
MIND-MAP



✓ MEMORIA FISSA

✗ Problema del CLUSTERING

✗ CANCELLAZIONE difficilissima

↳ usata solo quando non si
deve cancellare

 pos occup
contigue

LEFT-CH - RIGHT-SIB

~~BANDO~~~~BST select~~

BST invert

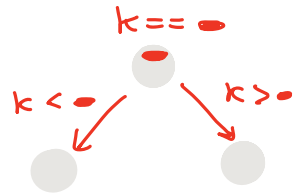
BST rep (bst-pic, bst-grande)

~~Complexity: PARAZIONE~~~~↳ estesa (n^2 nodi)~~~~↳ non estesa~~

```

      root
      ↓
Item search-R (link t, Key key) {
    if (t == NULL) return ITEMset NULL();
    int cmp = KEYcmp (KEYget (t->item), key);
    if (cmp == 0)
        return t->item;
    if (cmp < 0)
        return search-R (t->l, key);
    else
        return search-R (t->r, key);
}

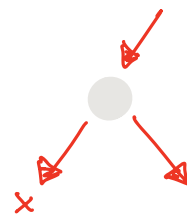
```



```

      root
      ↓
Item minR (link t) { // segue LEFT sempre
    if (t == NULL) return ITEMset null();
    if (t->l == NULL)
        return t->item;
    return minR (t->l);
}

```



```

void BST ins-leaf (BST bst, Item item) {

```

```

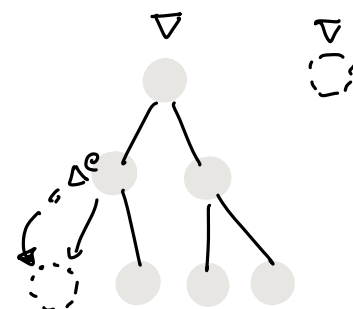
↓
3 bst → root = insertR (bst → root, item);
3
link insertR (link t, item item) {
    if (t == NULL) // ins in root
        return NEW node (item, NULL, NULL);

    int cmp = KEY cmp (KEY get (t → item), KEY get (item));

    if (cmp < 0)
        t → l = insertR (t → l, item);
    else
        t → r = insertR (t → r, item);

    return t;
}
3

```



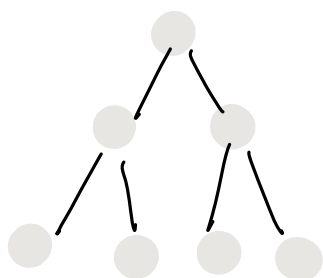
```

3
void visit IN_R (link t) {
    if (t == NULL) return;

    visit IN_R (t → l);
    printf (...);
    visit IN_R (t → r);
}
3

```

n° nodi | t_h

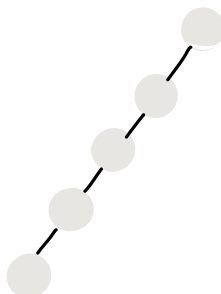


Completo

$$n = 2^h$$

$$h = \log_2 n$$

$$O(\log n)$$



Sbilanciato

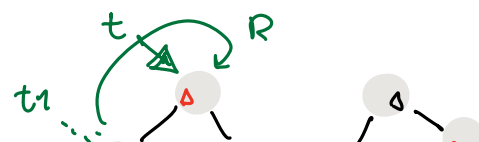
$$n = h + 1$$

$$h = n - 1$$

$$O(n)$$

⇒ importante il bilanciamento!

ROTAZIONI ⇒ BILANCIAMENTO




```
link rotR(link t) { ④(1)
```

```
link t1 = t->l;
```

```
t->l = t1->r;
```

```
t1->r = t;
```

```
return t1;
```

```
}
```



```
void BST insert-root (BST bst, Item item) {
```

```
    // inserimento con rotazioni
```

```
    bst->root = insert-root-R (bst->root, item);
```

```
}
```

```
link insert-root-R (link t, Item item) {
```

```
    if (t == NULL)
```

```
        return newnode (Item item, NULL, NULL);
```

```
    if (KEY cmp (KEY get (item), KEY get (t->item)) < 0) { // ins left
```

```
        |
```

```
        t->l = insert-root-R (t->l, item); // ins L
```

```
        t = rotR (t); // ruota R
```

```
        |
```

```
    else {
```

```
        |
```

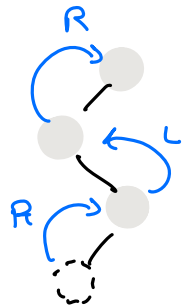
```
        t->r = insert-root-R (t->r, item); // ins R
```

```
        t = rotL (t); // ruota S
```

```
        |
```

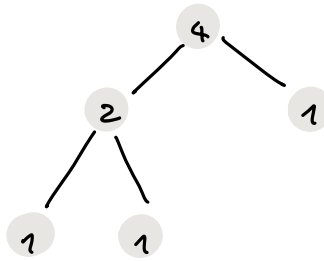
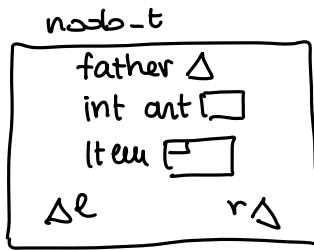
```
    return t;
```

```
}
```



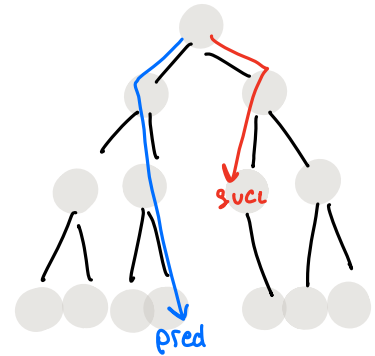
© BST ESZESO

Nuove info → move funz



SUCCESSORE : 1° passo dx, tutto a sx

PREDECESSORE : 1° passo sx, tutto a dx



```
void BST delete (BST bst, Key key) {
```

```
    ↓ bst->root = deleteR (bst->root, k);
```

```
    }
```

```
    link deleteR (link t, Key key) {
```

```
        if (t == NULL) return NULL;
```

```
        if (KEYcmp (key, KEYget (t->Item)) < 0)
            t->l = deleteR (t->l, key);
```

```
        else
```

```
            t->r = deleteR (t->r, key);
```

```
        (t->cnt)--;
```

```
        if (cmp == 0) {
```

```
            link tmp = t, p = t->p;
```

```
            t = joinLR (t->l, t->r);
```

```
            ↓ free tmp;
```

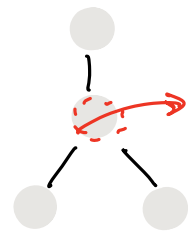
```
        }
```

```
        return t;
```

```
    }
```

```
    link joinLR (link a, link b) {
```

```
        if (a == NULL) return b;
```



```

if (b == Null) return a;

b = partR(b, 0);
b → l = a;
a → p = b;
b → cnt = a → cnt + b → r → cnt + 1;
return b;

```

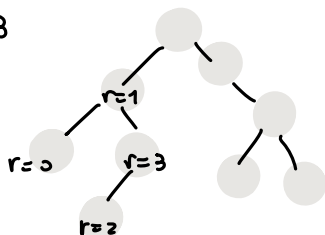
3

RANGO int r

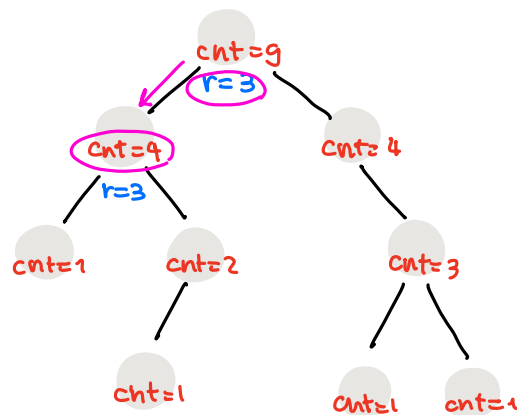
→ r=0 : chiave più piccola

→ r=n-1 : chiave più grande

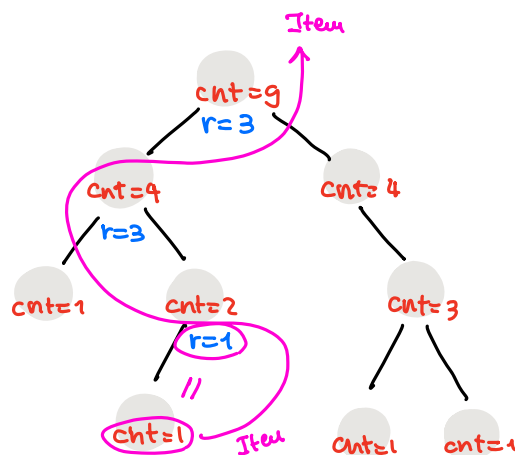
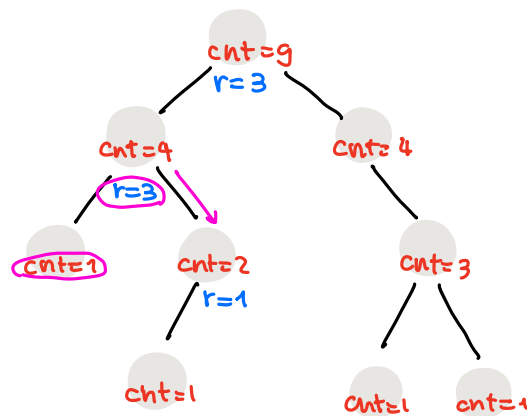
→ r=3



visit_IN ⇒ O(r)



SELECT-R ⇒ O(log n)



"RANGO"

Item BST select (BST bst, int r) {

↓ return selectR(bst → root, r);

3

Item selectR (link t, int r) {

if (t == Null) return ItemSetNull();

int num = t → l → cnt;

if (num > r)

return selectR(t → l, r);

if (num < r)

return selectR(t → r, r - num - 1);

if (num == r)

return t → item;

3

linr partR (linr t, ^{range r} int num-key)§

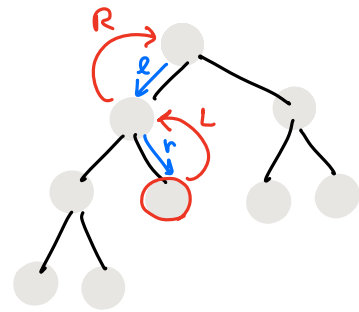
```

int num = t->l->cnt;
if (num > r)§
    t->l = partR(t->l, r);
    t = rotR(t);
}
if (num < r)§
    t->r = partR(t->r, r);
    t = rotL(t);
}
return t;

```

// sendo sx
// rot dx

// sendo dx
// rot sx



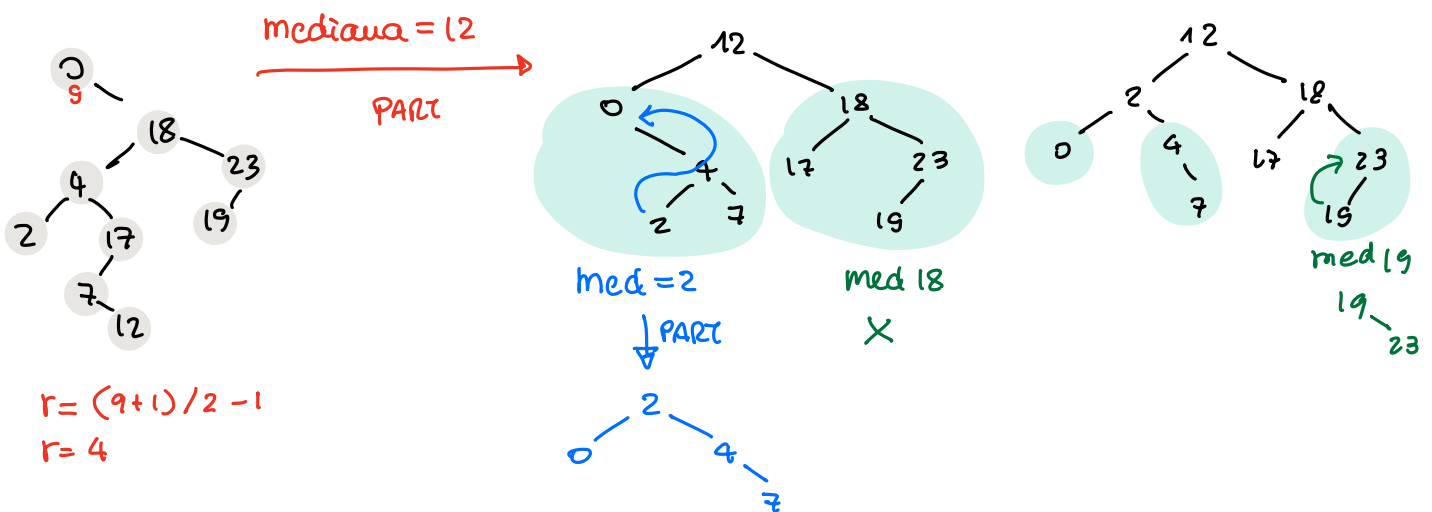
linr balanceR (linr t)§ // ^{root} WNGO LA MEDIANA!!!

if (t == null) return null;

^{range}
int num-key-mediana = (t->cnt + 1) / 2 - 1;
t = partR(t, num-key-mediana);

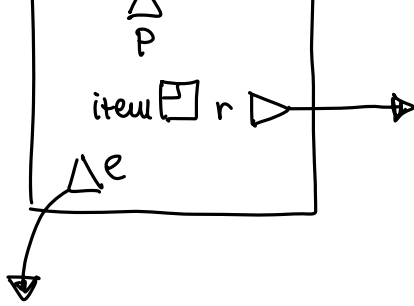
t->l = balanceR(t->l);
t->r = balanceR(t->r);
return t;

BILANCIAMENTO



LEFT-CHILD , RIGHT-SIBLING

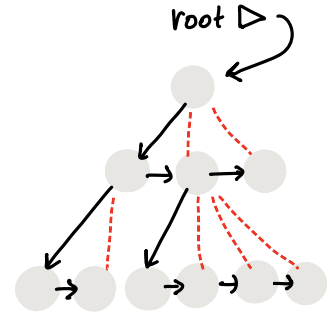




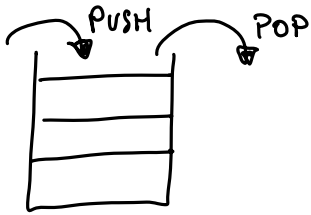
DATO UN ALBERO N-ARIO, CONTA TUTTI I SUOI NODI

```

int NT cnt ( NT nt ) {
  ↓ return cntR ( nt → root );
  }
  int cntR ( link t ) {
    ↓ if ( t == NULL ) return 0;
    ↓ return ( 1 + cntR ( t → l ) + cntR ( t → r ) );
    }
  }
  
```



- ✓ EFFICIENTE SPAZIO : solo 2 pt, indip dal grado
- ✓ COMODO per N-TREE
- ✗ INEFFICIENTE TEMPO $p \rightarrow \text{figlio}$ $\Theta(n)$

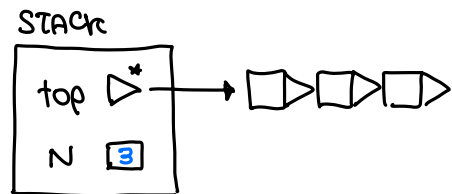


• STACK CON LISTA

```
typedef struct node * link;
```

```
struct stack {
```

```
    link top;
    int N;
};
```



```
void STACKpush(S s, Item item){
```

```
    // ins in testa
    link next = s->top;
    s->top = NEWnode(item, next);
    s->N++;
};
```

```
Item STACKpop(S s){
```

```
    // delete from testa
    Item it = s->top->item;
    link next = s->top->next;
    free(s->top);
    s->top = next;
    s->N--;
    return it;
};
```

CAPITOLO 14.0

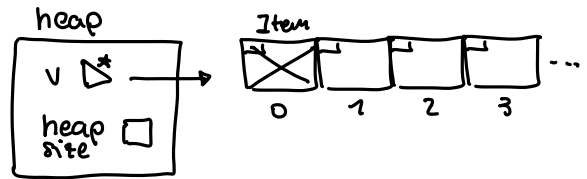
PRIORITY QUEUE

& HEAP

- **HEAP**: albero bin quasi completo

⇒ vettore

X								
0	1	2	3	4	5	6	7	8



struct heap {

Item * v;
int heapsize;
}

LEFT(i) = i * 2;

RIGHT(i) = i * 2 + 1;

PARENT(i) = i / 2;

Void HEAPIFY (Heap h, int i) {

// trasforma in HEAP(i) dove LEFT(i)

// e RIGHT(i) sono già HEAP

int ind_max;

int l = LEFT(i);

int r = RIGHT(i);

if ((l < h->heapsize) &&

ITEMcmp(h->v[l], h->v[i]) > 0)

↓ ind_max = l;

3

else

ind_max = i;

if ((r < h->heapsize) &&

ITEMcmp(h->v[r], h->v[i]) > 0)

↓ ind_max = r;

3

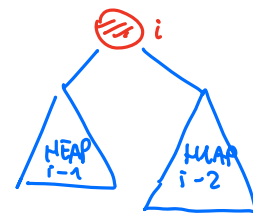
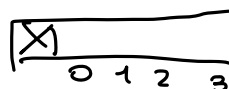
if (ind_max != i) {

SWAP(h, i, ind_max);

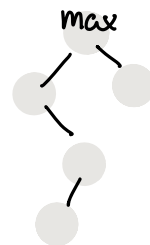
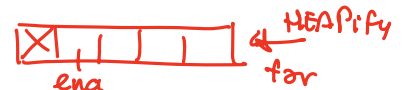
HEAPIFY(h, ind_max);

3

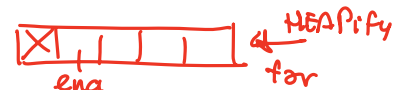
}



PER TRASFORMARE
TWO due PARTIRE
DAL FONDO!



PER TRASFORMARE
TWO due PARTIRE
DAL FONDO!



void HEAPbuild (heap h)

"HEAP BUILD"

```
    for (int i = (h->heapsize + 1) / 2; //scatto è  $\mathcal{O}$   
        i > 0; i++)  
        HEAPify(h, i);  
}
```


CAPITOLO 14.1

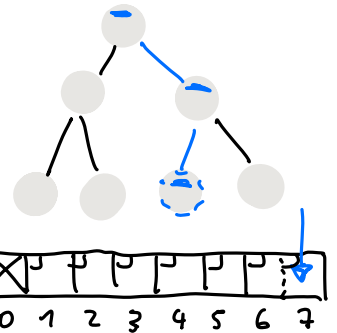
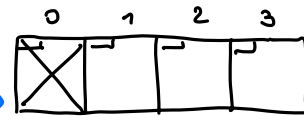
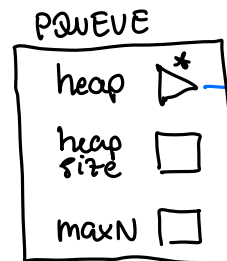
PRIORITY QUEUE

Implementata come HEAP di $\begin{matrix} \nearrow \text{DATI} \\ \searrow \text{INDICI "st"} \end{matrix}$

→ PQinsert(...) $\mathcal{O}(\log N)$

→ PQextract(...) $\mathcal{O}(\log N)$

```
struct pqueue {
    Item * heap;
    int heapsize;
    int maxN;
}
```



• void PQinsert (PQ pq, Item item) $\mathcal{O}(\log n)$

```
int i = pq->heapsize ++;
```

//CONFRONTI

```
while((i > 1) &&
```

```
    ITEMcmp( item, pq->heap[PARENT(i)]) > 0 ) {
```

```
    pq->heap[i] = pq->heap[PARENT(i)];
```

```
    i = PARENT(i);
```

//INS ITEM

```
pq->heap[i] = item;
```

• Item PQextractMax (PQ pq) $\mathcal{O}(\log n)$

```
Swap ( pq, 1, pq->heapsize );
```

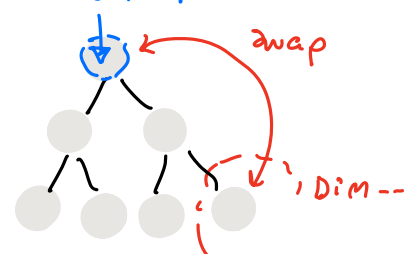
```
Item ext = pq->heap [ pq->heapsize ];
```

```
pq->heapsize --;
```

HEAPIFY (pq, 1);

```
return ext;
```

HEAPIFY



• void PQchange (PQ pq, Item item) {

//RICERCA $O(n)$!!!

// su vettore

int ind;

int trovato = 0;

for (int i = 0; i <= pq->heapsize && !trovato; i++) {

if (ITEMcmp (item, pq->heap[i]) == 0) {

trovato = 1;

ind = i;

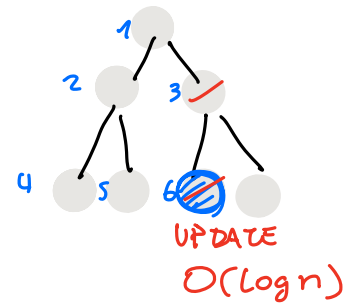
3

if (!trovato) return;

//UPDATE

...

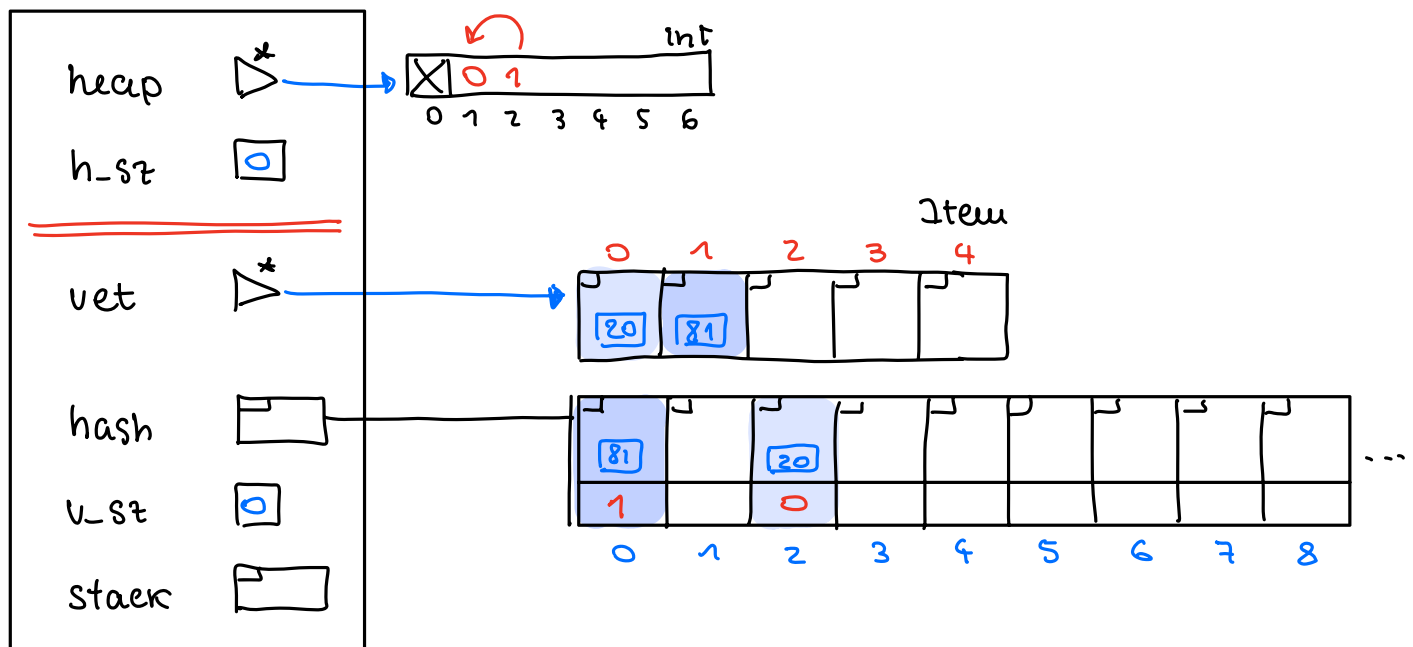
RICERCA $O(n)$



SOLO TEORICA

Coda a priorit  con **HEAP di INDICI** : Item troppo grossi

(1), 20 , (2), 81 , (3), 15 (4), 5 (5), 70



- 1) Ins in vet e in hash
 - 2) Ins in HEAP l'indice + UPDATE_HEAP
-) rep

- CICLO di HAMILTON da (v)

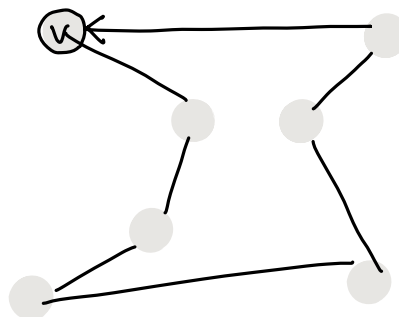
↳ semplice

↳ visita tutti i nodi
1 e 1 sola volta

visited o o o o o o

sol v - - - - - v

len V



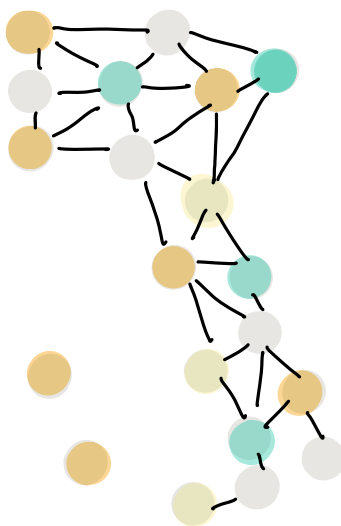
- COLORABILITÀ

n^o colori t.c. ogni v colore $\neq$ ai suoi w_i adiac

"Grafo PLANARE" = disegnato sul piano
e non si intersecano } mappe geog

↳ per il grafo planare servono al più 4 colori

MAPPA ITALIA



① STRUCT GRAFO

```
typedef struct Edge { int v, w; int wt; }
```

```
struct graph { struct st {
```

```

int V, E;
int ** adj;
ST tabv;
}

```

```

int N, maxN;
char ** v;
}

```

• INS V in ST File >> id_v id_w

0) INIZ di ST

1) *// read 1*
 $i = \text{ST searchbyname}(st, id_v);$
 $j = \text{ST searchbyname}(st, id_w);$

2) $\text{if}(i == -1) \text{// non c'è}$
 $\text{ST insert}(st, id_v);$

☹

3) INIZ di G

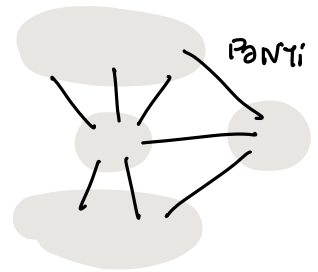
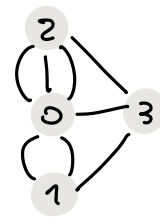
4) *// rewind (fp)* *// read 2*
 $\text{ins in } \text{MADJ}[i][j]$

5) return G

• MULTIGRAFO

In lettura 2: ins archi senza
 scartare quelli 3

↳ solo con *ladj* !!!



• ADJ vs MADJ

↳ *adj*

✓ GRAFI densi $S(n) = \Theta(V^2)$

✓ ACCESSO EFFICIENTE $T(n) = \Theta(1)$

✗ GRAFI poco densi → spreco spazio

↳ *ladj*

✓ gestisce il MULTIGRAFO

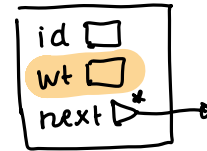
✓ GRAFO ORIENTATO: $S(n) = \mathcal{O}(V+E)$

• GRAFO NON ORIENTATO :

↳ GRAFI SPARSI

× ADJ $v-w$ $T(m) = O(m)$

× USO MEM : PESI wt



• CAMMINO SEMPLICE

visited o o o o o o

Scorro o MADJ o UADJ con RICORSIONE

trovato per interrompere

 COME LO FACCIO IO

• CAMMINO EULERO

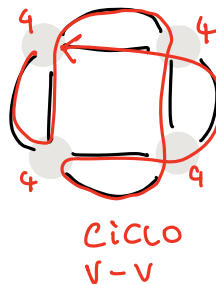
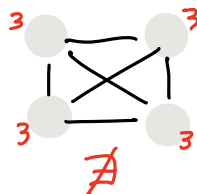
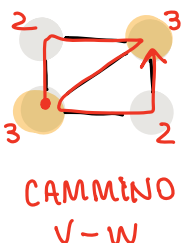
void GRAPH CammEulero (Graph G, int v, int w);

↳ cammino anche non semplice che li connette
attrav ogni e 1 e 1 sola volta

G non orient :

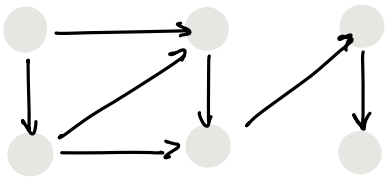
• $\exists$ ciclo Eul $\Leftrightarrow$ CONNESSO + $\forall$ grado PARI $v-v$

• $\exists$ Camm Eul $\Leftrightarrow$ CONNESSO + esat 2 v grado DISPARI



⑧ VISITA DEI GRAFI

- **PROFONDIITA'** : esaurisco la scoperta di un vertice **PRE-ORDER**



Vertici
0 1 2 3 4 5

ti	-1	-1	-1	-1	-1	-1
tf	-1	-1	-1	-1	-1	-1
cnt-p						
st	-1	-1	-1	-1	-1	-1
	0	1	2	3	4	5

ONLINE CONNECTIVITY

- **void GRAPHdfs (Graph G, int v) {**

```

    int time = 0;
    int *ti = malloc(G->V * sizeof(int));
    int *tf = _____;
    int *st = _____;
    for (int i=0; i < G->V; i++) {
        ↓ ti[i] = tf[i] = st[i] = -1;
    }

```

```

    dfsR(G, ti, tf, st, *time, v);

```

```

    DISPLAY result(...);
}

```

```

void dfsR( --- , p-time, src) {

```

```

    ti[src] = p-time;

```

```

    p-time++;

```

```

    DISPLAY color(...);

```

```

    for (t = G->adj[src]; t != NULL; t = t->next)

```

```

        if (ti[t->v] == -1) { // non ancora scop

```

```

            ↓ st[t->v] = src;

```

```

            ↓ dfsR(..., t->v);

```

```

        }

```

```

        else

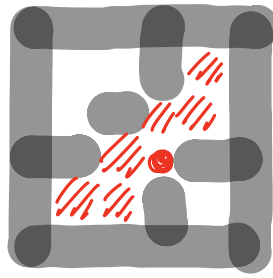
```

```

            DISPLAY color(...);
        }
    }
}

```

APPLICAZIONE: flood fill

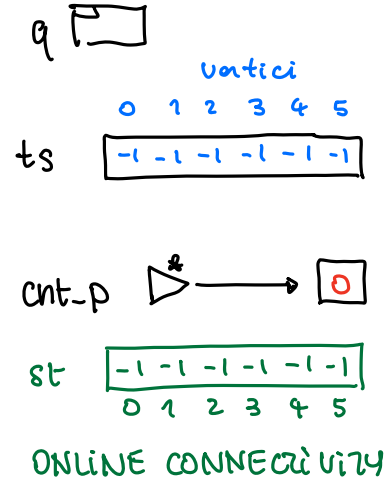


```

• void GRAPH bfs (Graph G, int v) {
    int time = 0;
    int * ts = malloc (G->V * sizeof(int));
    int * st = _____;
    for (int i = 0; i < G->V; i++)
        ts[i] = st[i] = -1;

    bfs_Q (G, *time, ts, st, v);

    DISPLAY sol (-1);
}
    
```



```

void bfs_Q (Graph G, int v) {
    int prev = v;
    Q q = Qinit();
    Qput(q, v);

    while (!Qempty(q)) {
        v = Qgetv(q);

        if (ts[v] == -1) {
            ts[v] = *time - p;
            (*time - p)++;
            st[v] = prev;
            prev = v;

            for (int i = 0; i < G->V; i++) {
                if (G->adj[v][i] != 0) {
                    if (st[i] == -1)
                        Qput(i);
                }
            }
        }
    }
}
    
```